Bài Tập Thực Hành

Kiến Trúc và Thiết Kế Phần Mềm

Nguyễn Trọng Tiến

SOFTWARE ARCHITECTURE

Buổi 5 — SERVICE-BASED ARCHITECTURE

Bài toán: Mini Food Ordering System

Một công ty muốn xây dựng hệ thống đặt món ăn nội bộ cho nhân viên (giống ShopeeFood mini).

- Yêu cầu chức năng:
  1. Quản lý món ăn:
     - Xem danh sách món ăn
     - Thêm / sửa / xóa món ăn
  2. Quản lý người dùng:
     - Đăng ký / đăng nhập
     - Phân quyền (USER / ADMIN)
  3. Đặt món:
     - Thêm món vào giỏ hàng
     - Tạo đơn hàng
  4. Thanh toán (giả lập):
     - Chọn phương thức thanh toán (COD / Banking)
     - Cập nhật trạng thái đơn hàng
  5. Thông báo
     - Khi đặt hàng thành công → gửi thông báo (console log hoặc REST call)
- **Yêu cầu kiến trúc:**

  Áp dụng Service-Based Architecture:
  - Mỗi chức năng = 1 service riêng biệt (Spring Boot)
  - Giao tiếp qua REST API (HTTP)
  - Có thể dùng API Gateway (optional)

Phân công 5 người:

- Người 1– Frontend (ReactJS)

  UI:
  - Login/Register
  - Danh sách món
  - Giỏ hàng
  - Đặt hàng

  Gọi API từ các service

  Tech:
  - ReactJS + Axios

- Người 2 – User Service (Spring Boot)
  API:
    - POST /register
    - POST /login
    - GET /users

  Yêu cầu:
    - JWT đơn giản (optional)
    - Lưu memory hoặc H2
- Người 3 – Food Service
  API:
    - GET /foods
    - POST /foods
    - PUT /foods/{id}
    - DELETE /foods/{id}

  Yêu cầu:
    - Không cần auth phức tạp
    - Seed sẵn dữ liệu
- Người 4 – Order Service
  API:
    - POST /orders
    - GET /orders

  Khi tạo order:
    - Gọi Food Service để lấy thông tin món
    - Gọi User Service để validate user
- Người 5 – Payment + Notification Service
  API:
    - POST /payments

  Khi thanh toán:
    - Update trạng thái order (gọi Order Service)
    - Gửi notification

  Notification:
    - Gọi API hoặc log:
      User A đã đặt đơn #123 thành công

Mô hình triển khai (LAN)

- Mỗi người chạy service trên máy riêng:
  - 192.168.?.?:8081 → User Service
  - 192.168. ?.?:8082 → Food Service

- 192.168. ?.?:8083 → Order Service
- 192.168. ?.?:8084 → Payment Service

- Frontend gọi trực tiếp

Cấu hình CORS + IP thật (KHÔNG dùng localhost chéo máy)

Kịch bản Test (BẮT BUỘC DEMO)

1. User đăng ký + login
2. Xem danh sách món
3. Thêm vào giỏ → tạo order
4. Thanh toán
5. Nhận thông báo

Bonus (nếu còn thời gian)

1. API Gateway (Spring Cloud Gateway)
2. Load balancing (round robin giả lập)
3. Retry khi service fail
4. Logging tập trung

Tiêu chí chấm điểm

Tiêu chí	Điểm
Đúng kiến trúc Service-Based	3
API hoạt động	2
Giao tiếp giữa services	2
Frontend chạy mượt	1.5
Demo hoàn chỉnh	1

Giai đoạn 2 (Homework):

- Dockerize
  - Mỗi service = 1 container
  - docker-compose chạy toàn hệ thống
- Deploy local server (1 máy)

Giai đoạn 3: (Optional)

- Dockerize
  - Chạy hệ thống trên 1 server thật (VPS hoặc máy lab)

Buổi 6 — EVENT-DRIVEN ARCHITECTURE

Bài toán: Movie Ticket System

Một hệ thống đặt vé xem phim với yêu cầu xử lý bất đồng bộ (asynchronous) để đảm bảo hệ thống scalable.

❖ **Yêu cầu chức năng:**

- Quản lý phim:
  - Xem danh sách phim
  - Thêm / sửa phim
- Quản lý người dùng:
  - Đăng ký / đăng nhập
- Đặt vé:
  - Chọn phim + số ghế
  - Tạo booking
- Thanh toán:
  - Thanh toán vé (giả lập)
  - Cập nhật trạng thái booking
- Thông báo
  - Gửi thông báo khi đặt vé thành công

❖ **Yêu cầu kiến trúc:**

Áp dụng Event-Driven Architecture:

- Các service KHÔNG gọi trực tiếp nhau
- Giao tiếp qua Message Broker (Kafka / RabbitMQ / Redis PubSub)

Luồng event chính:

User → Booking Service → (Publish Event)
→ Payment Service (Consume)
→ Notification Service (Consume)

Danh sách Event:

Event	Mô tả
USER_REGISTERED	Người dùng đăng ký
BOOKING_CREATED	Tạo booking
PAYMENT_COMPLETED	Thanh toán xong
BOOKING_FAILED	Thanh toán thất bại

Phân công 5 người:

❖ Người 1– Frontend (ReactJS)
UI:

- Login/Register
- Danh sách phim
- Đặt vé

Gọi API chỉ vào 1 service (Gateway hoặc Booking Service)
Không gọi trực tiếp tất cả service

❖ Người 2 – User Service (Spring Boot)
API:
- POST /register
- POST /login

Khi đăng ký:
- Publish event: USER_REGISTERED

❖ Người 3 – Movie Service
API:
- GET /movies
- POST /movies

Yêu cầu:
- Không cần event phức tạp

❖ Người 4 – Booking Service (CORE)
API:
- POST /bookings
- GET / bookings

Khi tạo booking:
- Publish event: BOOKING_CREATED

KHÔNG xử lý payment trực tiếp

❖ Người 5 – Payment + Notification Service
Payment:
- Listen: BOOKING_CREATED
- Xử lý: Random success/fail
- Publish:
  PAYMENT_COMPLETED hoặc BOOKING_FAILED

Notification:
- Listen: PAYMENT_COMPLETED
- Output: "Booking #123 thành công!"

Notification:
- Gọi API hoặc log:
  User A đã đặt đơn #123 thành công

Mô hình triển khai trên LAN:

Service	IP
User	**192.168.?.?:8081**
Movie	**192.168.?.?:8082**
Booking	**192.168.?.?:8083**
Payment	**192.168.?.?:8084**
Frontend	**192.168.?.?:8085**

Broker chạy riêng:

Kafka / RabbitMQ: 192.168.?.?:9092

Kịch bản Test (BẮT BUỘC DEMO)

1. User đăng ký → log event
2. Chọn phim→ đặt vé
3. Payment xử lý
4. Notification hiển thị kết quả

Bonus (nếu làm nhanh)

1. Dead Letter Queue
2. Retry mechanism
3. Event log (lưu lịch sử event)
4. Dashboard realtimeAPI Gateway (Spring Cloud Gateway)

Tiêu chí chấm điểm

Tiêu chí	Điểm
Đúng Event-Driven	3
Publish/Consume đúng	2.5
Flow hoạt động end-to-end	2
Không gọi trực tiếp service	1.5
Demo + log rõ ràng	1

Giai đoạn 2 (Homework):

- Dockerize
  - Mỗi service = 1 container
  - docker-compose chạy toàn hệ thống
- Deploy local server (1 máy)

Giai đoạn 3: (Optional)

- Chạy hệ thống trên 1 server thật (VPS hoặc máy lab)

Buổi 7 — SPACE-BASED ARCHITECTURE

Bài toán: Hệ thống Flash Sale (Bán hàng sốc – chịu tải cao)

Một hệ thống bán hàng flash sale (giống Shopee/Lazada) cần:

- Chịu tải cao (1000+ request/s)
- Tránh nghẽn database
- Xử lý nhanh (low latency)

❖ **Yêu cầu 5 chức năng chính:**

- Xem danh sách sản phẩm
- Xem chi tiết sản phẩm
- Thêm vào giỏ hàng
- Đặt hàng (checkout)
- Giảm tồn kho (real-time)

❖ **Yêu cầu kiến trúc:**

Áp dụng Space-Based Architecture:

❖ **Nguyên lý:**

- Hạn chế DB (tránh bottleneck)
- Dữ liệu nằm trong Memory Grid (Data Grid)
- Xử lý tại Processing Unit (PU)

❖ **Thành phần chính**

- Processing Unit (PU) = Service xử lý + cache local
- Data Grid = Redis / Hazelcast (chia sẻ dữ liệu RAM)
- Messaging (optional)

Phân công 5 người:

❖ Người 1– Frontend (ReactJS)

UI:

- Danh sách sản phẩm
- Giỏ hàng
- Đặt hàng

Gọi API vào Processing Unit (PU)

❖ Người 2 – Product Processing Unit (PU1)

API:

- GET /products
- GET /products/{id}

Data:

- Load từ Data Grid (Redis)

Không đọc DB trực tiếp

❖ Người 3 – Cart Processing Unit (PU2)

API:

- POST /cart/add
- GET /cart

Data:

- Lưu trong Data Grid (session/cart)

❖ Người 4 – Order Processing Unit (PU3)

API:

- POST / checkout

Xử lý:

- Lấy cart từ Data Grid
- Tạo order
- Publish event (optional)

❖ Người 5 – Inventory Processing Unit (PU4)

API:

- GET /stock/{productId}

Xử lý:

- Khi checkout
  - Giảm tồn kho trực tiếp trên Data Grid

Không gọi DB

Mô hình triển khai trên LAN:

Service	IP
Redis (Data Grid)	**192.168.?.?:6379**
PU1 – Product	**192.168.?.?:8081**
PU2 – Cart	**192.168.?.?:8082**
PU3 – Order	**192.168.?.?:8083**
PU4 – Inventory	**192.168.?.?:8084**
Frontend	**192.168.?.?:3000**

Luồng xử lý chính

❖ Luồng đặt hàng

1. User chọn sản phẩm → add to cart
2. Cart lưu vào **Data Grid (Redis)**
3. User checkout
4. Order PU:
   - Lấy cart từ Redis

- Gọi Inventory (hoặc trực tiếp Redis)
- Giảm stock

5. Trả kết quả ngay (KHÔNG chờ DB)

Kịch bản Test (BẮT BUỘC DEMO)

1. Load danh sách sản phẩm từ Redis
2. Add to card
3. Checkout
4. Stock giảm ngay lập tức
5. Không bị chậm khi nhiều request

Bonus (nếu làm nhanh)

1. Dùng Hazelcast thay Redis
2. Implement locking (SETNX)
3. Thêm Queue xử lý async
4. Simulate load test (Postman Runner)

Tiêu chí chấm điểm

Tiêu chí	Điểm
Đúng Space-Based	3
Không phụ thuộc DB	2.5
Dùng Data Grid đúng	2
Flow nhanh, không nghẽn	1.5
Demo scale (clone PU)	1

Buổi 8: ORCHESTRATION-DRIVEN SOA

Bài toán: Travel Booking System

Xây dựng hệ thống đặt tour:

- Người dùng chọn tour
- Đặt tour
- Thanh toán
- Nhận xác nhận

❖ **5 chức năng chính**

1. Quản lý người dùng
   - Đăng ký / đăng nhập
2. **Quản lý tour**
   - Xem danh sách tour
   - Chi tiết tour
3. **Đặt tour**
   - Tạo booking
4. **Thanh toán**
   - Thanh toán booking
5. **Xác nhận**
   - Gửi thông báo booking thành công

Yêu cầu kiến trúc:

Áp dụng Orchestration-Driven SOA

Nguyên lý:

- Có Orchestrator Service (trung tâm)
- Các service khác:
  - KHÔNG gọi nhau trực tiếp
  - Chỉ nhận lệnh từ Orchestrator

Thành phần hệ thống

Thành phần	Vai trò
Orchestrator	Điều phối toàn bộ flow
User Service	Quản lý user
Tour Service	Quản lý tour
Booking Service	Tạo booking
Payment Service	Thanh toán

Phân công 5 người

- Người 1 – Frontend (ReactJS)
  - UI:
    - Login
    - Xem tour
    - Đặt tour

Gọi:

  - Chỉ gọi Orchestrator
    Không gọi service khác

- Người 2 – Orchestrator Service
  - API:
    - POST /book-tour

Flow trong Orchestrator:

1. Validate user (User Service)
2. Lấy thông tin tour (Tour Service)
3. Tạo booking (Booking Service)
4. Gọi Payment Service
5. Trả kết quả về Frontend

Tất cả đều là REST call

- Người 3 – User Service
  - API:
    - POST /login
    - GET /users/{id}

- Người 4 – Tour Service
  - API:
    - GET /tours
    - GET /tours/{id}

- Người 5 – Booking + Payment Service

Booking:

  - API:
    - POST /bookings

Payment:

  - API:
    - POST /payments

Logic:

- Random success/fail

Triển khai trên LAN

Service	**IP**
Orchestrator	192.168.1.10:8080
User	192.168.1.11:8081
Tour	192.168.1.12:8082
Booking	192.168.1.13:8083
Payment	192.168.1.14:8084
Frontend	192.168.1.15:3000

Flow chi tiết

Flow đặt tour

1. Frontend → Orchestrator
2. Orchestrator:
    - gọi User Service
    - gọi Tour Service
    - gọi Booking Service
    - gọi Payment Service
3. Trả kết quả về Frontend

Buổi 09: MICROSERVICES ARCHITECTURE

Bài toán: E-Commerce Mini

Xây dựng hệ thống bán hàng đơn giản (giống Shopee mini):

- Người dùng mua sản phẩm
- Tạo đơn hàng
- Thanh toán

❖ **5 chức năng chính**

1. Quản lý người dùng
   - Đăng ký / đăng nhập
2. Quản lý sản phẩm
   - Xem danh sách sản phẩm
   - Thêm / sửa sản phẩm
3. Giỏ hàng
   - Thêm / xóa sản phẩm
4. Đặt hàng
   - Tạo đơn hàng từ giỏ
5. Thanh toán
   - Thanh toán đơn hàng

Yêu cầu kiến trúc:

Áp dụng Microservices Architecture

Nguyên tắc:

- Mỗi domain = 1 service
- Service có database riêng
- Giao tiếp qua REST API
- Không share DB

Danh sách Microservices

Service	**Chức năng**
User Service	User
Product Service	Product
Cart Service	Cart
Order Service	Order

Payment Service	Payment

Phân công 5 người

❖ Người 1 – Frontend (ReactJS)

UI:

- Login/Register
- Danh sách sản phẩm
- Giỏ hàng
- Checkout

Gọi:

- API Gateway (hoặc gọi trực tiếp service)

❖ Người 2 – User Service

API:

- POST /register
- POST /login
- GET /users

DB riêng:

- users table

❖ Người 3 – Product Service

API:

- GET /products
- POST /products
- PUT /products/{id}

DB riêng:

- products table

❖ Người 4 – Cart Service

API:

- POST /cart/add
- GET /cart/{userId}
- DELETE /cart/item

Logic:

- Lưu cart theo user

Không lưu product detail (chỉ productId)

❖ Người 5 – Order + Payment Service

Order:

API:

- POST /orders
- GET /orders

Flow:

- Lấy cart từ Cart Service
- Gọi Product Service để lấy giá
- Tạo order

Payment:

API:

- POST /payments

Logic:

- Update trạng thái order

Triển khai trên LAN

Service	IP
User	192.168.?.?:8081
Product	192.168.?.?:8082
Cart	192.168.?.?:8083
Order	192.168.?.?:8084
Frontend	192.168.?.?:3000
User	192.168.?.?:8081

Luồng xử lý chính

Flow đặt hàng

1. User login
2. Xem sản phẩm
3. Add to cart
4. Checkout:
   - Cart Service → lấy cart
   - Product Service → lấy giá
   - Order Service → tạo order
5. Payment → update trạng thái

Buổi 10: HYBRID (EVENT-DRIVEN + MICROSERVICES)

Bài toán: Food Delivery System

Hệ thống giống GrabFood/ShopeeFood mini:

- Cần phản hồi nhanh (REST)
- Nhưng xử lý hậu trường (async) → Event

❖ **5 chức năng chính**

1. Quản lý người dùng
   - Đăng ký / đăng nhập
2. Xem món ăn
   - Danh sách món
   - Chi tiết món
3. Đặt hàng
   - Tạo order
   - Xem order
4. Thanh toán
   - Thanh toán đơn hàng
5. Thông báo
   - Gửi thông báo khi order thành công

Yêu cầu kiến trúc: Hybrid

Microservices (REST – synchronous)

- Frontend → API Gateway → Service
- Dùng cho:
  - Login
  - Get data (foods, orders)

Event-Driven (async)

- Dùng cho:
  - Payment
  - Notification
  - Order processing hậu kỳ

Nguyên lý thiết kế

Loại xử lý	Dùng gì
Cần response ngay	REST
Không cần response ngay	Event

Luồng hệ thống (QUAN TRỌNG)
Flow chính:

1. User → Frontend → API Gateway
2. Gateway → Order Service (REST)
3. Order Service:
   - Lưu order
   - Publish event:

ORDER_CREATED

4. Payment Service (consume event):
   - Xử lý thanh toán
   - Publish:

PAYMENT_SUCCESS

5. Notification Service:
   - Gửi thông báo

Danh sách Event

Event	Mô tả
ORDER_CREATED	Tạo đơn
PAYMENT_SUCCESS	Thanh toán thành công
PAYMENT_FAILED	Thanh toán thất bại

Phân công 5 người

❖ Người 1 – Frontend (ReactJS)

UI:

- Login/Register
- Danh sách món
- Đặt hàng

Gọi:

- API Gateway (REST)

Không biết event phía sau

❖ Người 2 – API Gateway

Route:

- /api/users → User Service
- /api/foods → Food Service
- /api/orders → Order Service

Có thể dùng:

- Spring Cloud Gateway

❖ Người 3 – User + Food Service

User:

- POST /register
- POST /login

Food:

- GET /foods

Pure REST (không event)

❖ Người 4 – Order Service (CORE)

- API:
  - POST /orders
  - GET /orders

Khi tạo order:

- Lưu DB
- Publish event:

ORDER_CREATED

Không xử lý payment trực tiếp

❖ **Người 5 – Payment + Notification Service**

Payment Service:

- Consume:

ORDER_CREATED

- Xử lý:
  - Random success/fail
- Publish:

PAYMENT_SUCCESS hoặc PAYMENT_FAILED

Notification Service:

- Consume:

PAYMENT_SUCCESS

- Output:
  "*Đơn hàng #123 đã thanh toán thành công*!"

Triển khai trên LAN

Service	IP
Gateway	192.168.1.10:8080
User/Food	192.168.1.11:8081
Order	192.168.1.12:8082
Payment	192.168.1.13:8083
Frontend	192.168.1.14:3000
Kafka/RabbitMQ	192.168.1.100:9092

Kịch bản demo

1. User login
2. Xem món
3. Đặt hàng (REST)
4. Payment chạy ngầm (Event)
5. Notification hiển thị

----------Hết----------